

Powering FloodNet: Optimizing Sensor Reporting for Battery Life

15-Minute Reporting Cuts Sensor Energy Use 41% and Extends Battery Life from 49 to 84 Days

Nicholas Reinoso,¹ Charlie Mydlarz,² Andrea I. Silverman^{2,3}

1. Undergraduate Summer Research Program, NYU Tandon School of Engineering 2. Center for Urban Science and Progress, NYU Tandon 3. Department of Civil and Urban Engineering, NYU Tandon

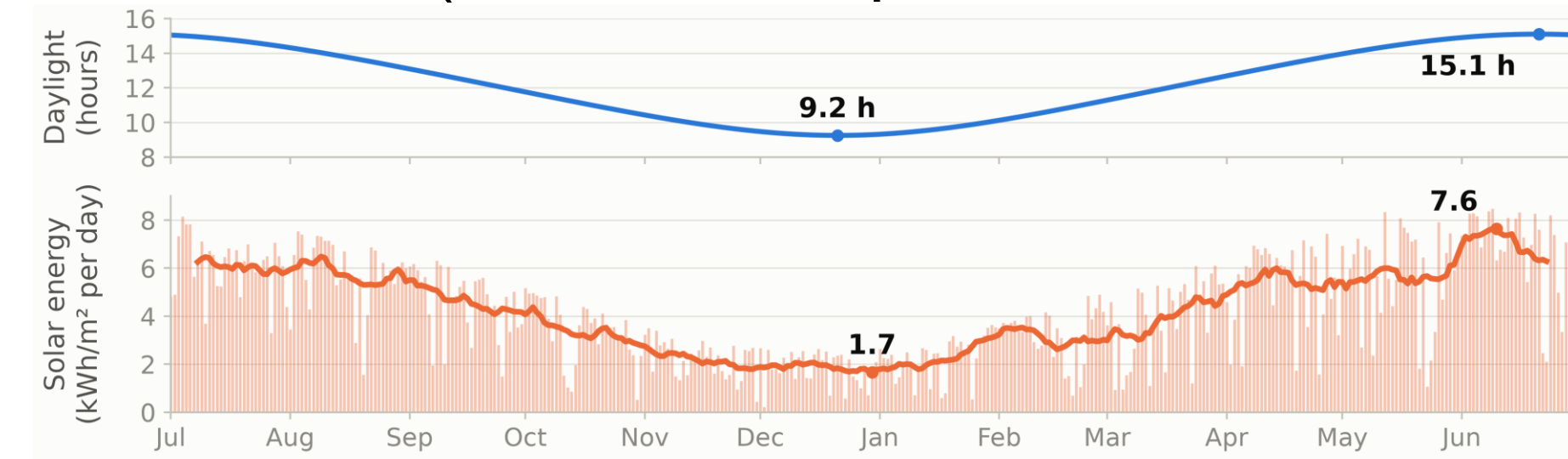
Project Supervisor: Andrea I. Silverman Research Staff: Charlie Mydlarz

Acknowledgments: The author thanks the NYU Tandon School of Engineering's Office of Undergraduate Academics for generous funding of this project. Thanks also to Praneeth Challagonda, Chris Hoyte, Amanpreet Kaur, Polly Pierone, and Bea Steers.

Abstract

FloodNet is a street-level flood monitoring network with 450 sensors across New York City's five boroughs. Each solar-powered unit measures water level every minute with an ultrasonic sensor and reports over cellular at a remotely settable interval (default 1 minute). In winter, shorter days and building shading cut solar harvest below what a shaded sensor consumes: batteries drain over weeks and sensors brown out. To measure each reporting rate's real cost, a sensor's current draw was recorded with a Qoitech Otii Arc power analyzer at 1-, 15-, and 30-minute intervals. Slowing reporting from 1 to 15 minutes cut average draw by 41%, stretching a full charge from 49 to 84 days with no solar input. Slowing to 30 minutes saved nothing more: a 900-second MQTT keepalive wakes the modem mid-interval regardless. Fifteen-minute reporting therefore delivers the most live data at the lowest measured power. A server-side policy is proposed: 15-minute reporting below a battery voltage threshold, with 1-minute reporting restored when storms threaten.

Figure 1: Daylight and daily solar energy (kWh/m² per day) near NYC, July 2025 to June 2026 (solar data: Open-Meteo, CC BY 4.0)



Testing

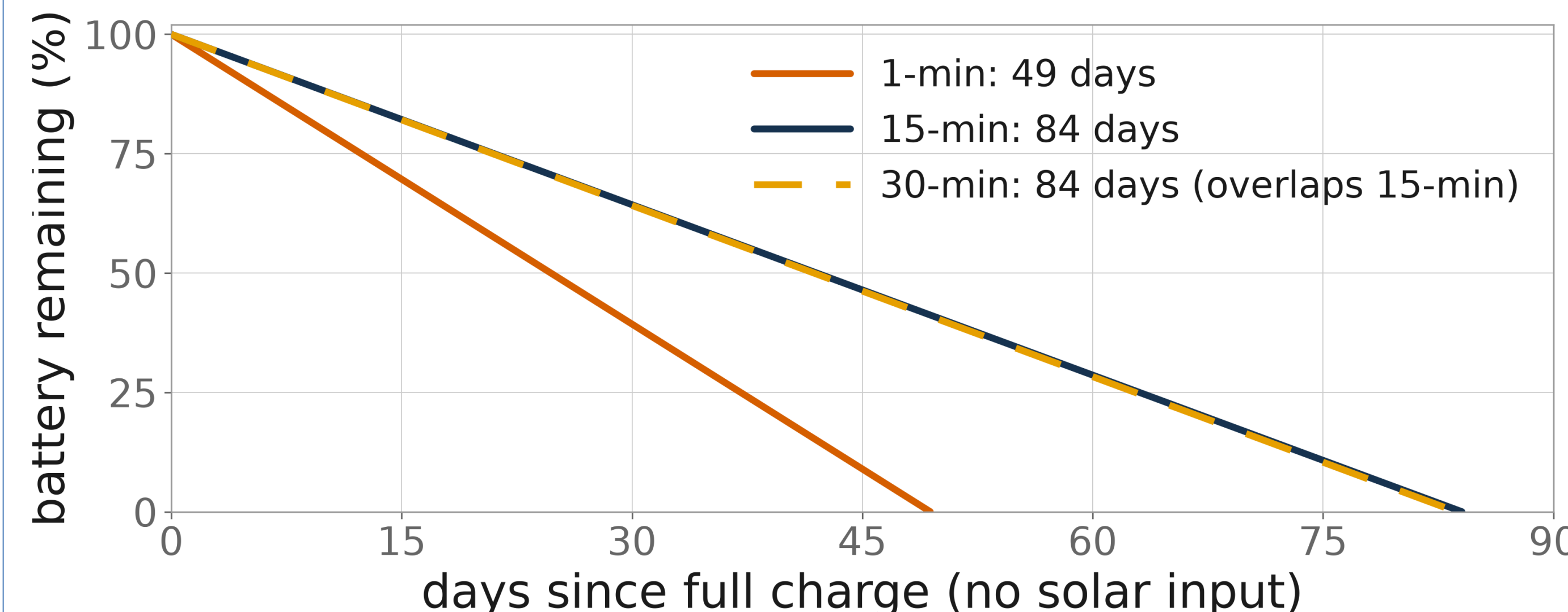
Testing used a Qoitech Otii Arc power analyzer recording current at 4,000 samples per second at the battery's 3.60 V nominal voltage. The 1-, 15-, and 30-minute schedules were captured for 63, 23, and 16 hours, respectively. The Arc's UART log matched each current spike to a logged sensor event.



Live Otii Arc view during bench testing: current, voltage, and VBUS traces, with the sensor's time-synced UART log below (current sampled at 4,000 samples per second).

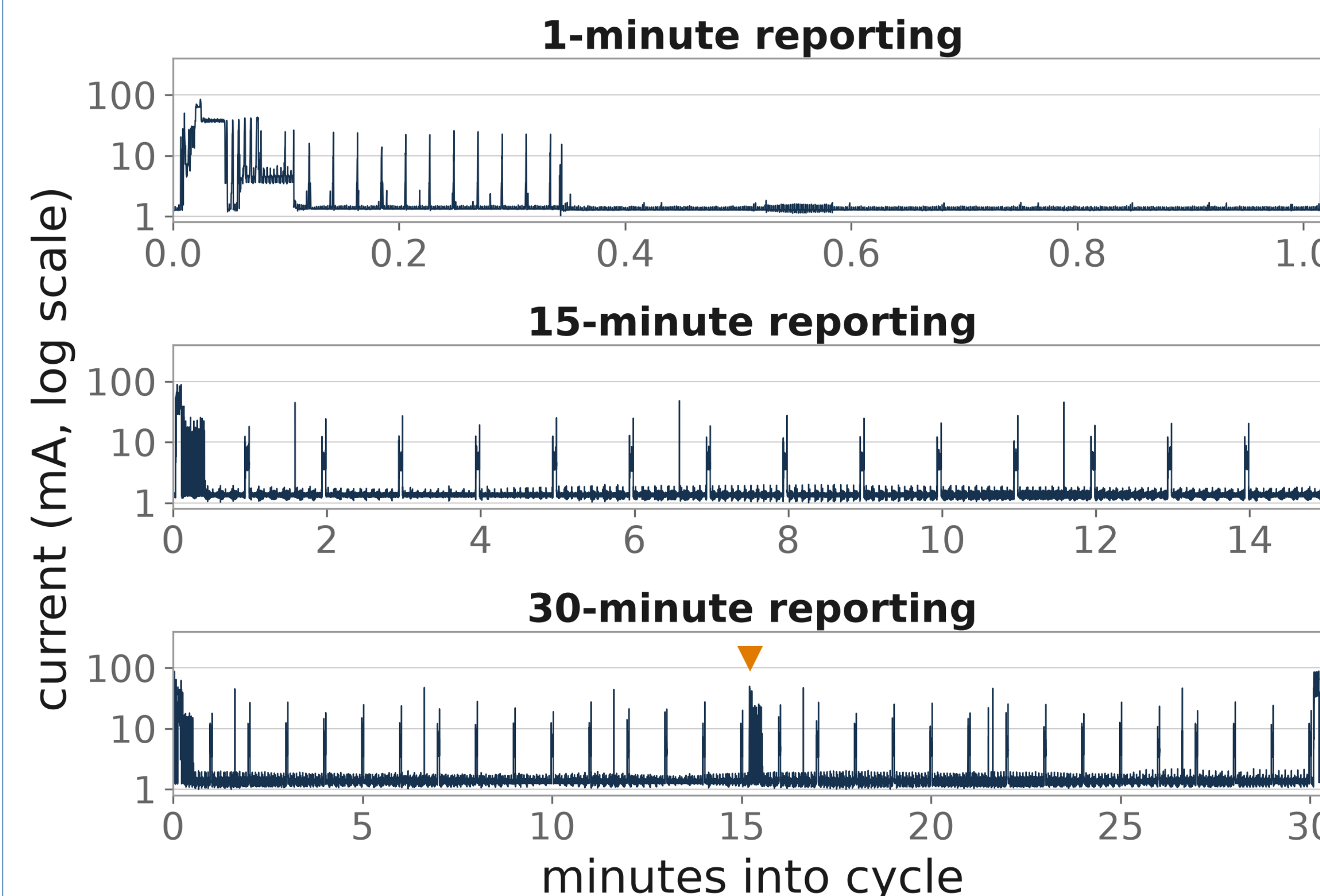
Results

Figure 2: Predicted state of charge at 1-, 15-, and 30-minute reporting on the Samsung INR18650-35E cell, no solar input



Slowing reporting from 1-minute to 15-minute intervals cut average current draw by 41%, from 2.82 to 1.66 mA. Slowing to 30 minutes gave no measurable additional saving over 15-minute reporting. With the Samsung INR18650-35E lithium-ion cell (3,350 mAh), the measured currents project a full charge, with no solar input, lasting 49 days at 1-minute reporting and 84 days at 15- and 30-minute reporting (Figure 2).

Figure 3: Measured current over one report cycle at 1-, 15-, and 30-minute reporting (log scale, 4,000 samples per second)

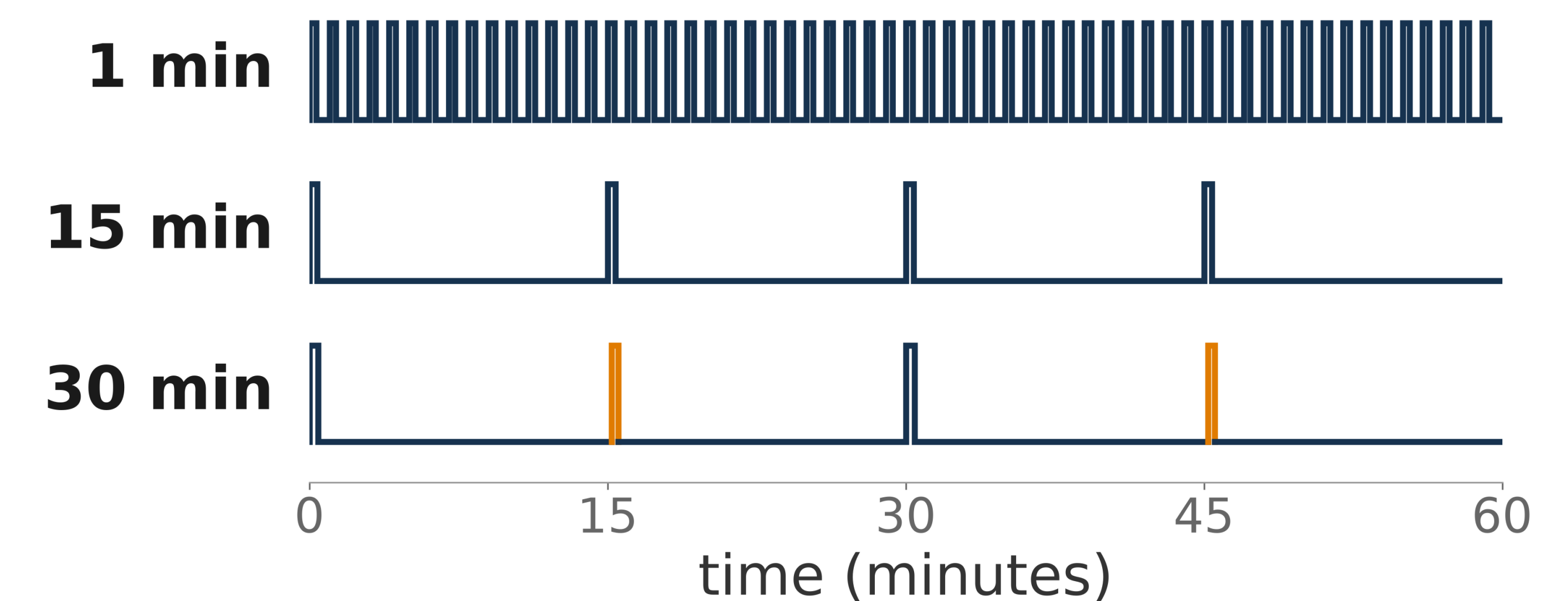


The radio wakes only once per interval to send its stored readings, but the MQTT connection times out after 900 seconds without traffic, so the 30-minute schedule adds a mid-interval keepalive ping (orange triangle, Figure 3).

Figure 4: Radio wakes per hour at 15- and 30-minute reporting



Figure 5: Radio duty cycle over one hour at each reporting rate



Because of the keepalive ping (Figure 4), 15- and 30-minute reporting produce the same four radio wakes per hour and nearly identical radio-active time, so their energy use is equivalent (Figure 5). Radio wakes draw the sensor's highest instantaneous current (transmit peaks above 100 mA against a 1.26 mA idle floor). The added keepalive wake cancels the saving that halving the upload rate should provide.

Future Work

Proposals: (1) a server-side policy: 1-minute reporting above a battery voltage threshold, 15-minute below, with 1-minute reporting restored when storms or nearby flooding threaten; (2) raising the 900 s MQTT keepalive toward the Azure IoT Hub limit of about 1,177 s and measuring energy at longer keepalive intervals; (3) shortening the 250 ms pause the firmware leaves between queued records during an upload.